

CASE STUDY REPORT

AI Powered Video Conferencing Website (Java + Spring Boot + Python + AI)

1. Project Overview

This project is an AI-powered Video Conferencing Website developed using Java, Spring Boot, and Python AI services. The main purpose of the project is to provide an online meeting platform where users can create meetings, join meetings, and communicate through video, audio, and chat. Along with normal video call features, this system also provides AI-based support such as automatic meeting transcription and meeting summary generation.

This project is designed for organizations, students, and teams who need secure video meetings with additional productivity features.

2. Problem Statement

In online meetings, users face many challenges such as:

- Difficulty in taking notes during meetings
- Missing important discussion points
- No meeting summary after call ends
- Time waste in writing meeting minutes manually

Most video conferencing platforms provide only calling features, but do not support AI productivity tools. Therefore, a smart video conferencing platform is required to improve efficiency and save time.

3. Project Objective

The main objectives of this project are:

- Provide secure login and registration for users
- Allow users to create and join video meetings
- Enable real-time video/audio communication
- Provide meeting chat functionality
- Generate AI-based transcript and summary of meetings
- Store meeting history for future access

This system improves user experience by reducing manual work after meetings.

4. Technologies Used

Backend:

- Java
- Spring Boot
- Spring MVC
- Spring Security
- REST APIs
- JWT Authentication

Real-Time Communication:

- WebRTC (for video/audio calling)
- WebSocket (for signaling and chat)

AI Implementation:

- Python (Flask/Fast API)
- Django
- Speech-to-text module for transcription
- AI model for summarization

Database:

- MySQL (for storing user and meeting data)

Tools:

- Postman
- Swagger
- GitHub
- Maven

5. System Architecture

The project follows a modular architecture where each part has its own responsibility.

Main Components:

1. Frontend Website – User interface for login, meeting dashboard, video call screen
2. Spring Boot Backend – Handles authentication, meeting management, and database operations
3. WebRTC Module – Handles video/audio connection between participants
4. Python AI Service – Converts meeting audio into text and generates meeting summary
5. Database – Stores users, meeting history, transcript, and summary

6. Authentication & Security

Security is implemented using Spring Security and JWT Token Authentication.

Flow:

1. User registers and logs into the system
2. Spring Boot generates JWT token
3. Token is used for every request
4. Backend validates token before allowing access

This ensures only authorized users can create or join meetings.

7. Meeting Workflow

Meeting Creation:

- Host logs in
- Host creates meeting
- System generates a unique meeting code
- Meeting details are saved in database

Meeting Joining:

- Participants enter meeting code
- Backend validates meeting code
- User joins meeting room
- Video/audio call starts using WebRTC

Meeting End:

- Host ends meeting
- Meeting data is saved in database
- Transcript and summary are generated

8. AI Features

This project includes AI features using Python service.

AI Features:

1. Live Transcription – Audio from meeting is processed and converted into text transcript
2. Meeting Summary – Transcript is analyzed and AI generates a short summary
3. Keyword Extraction (Optional) – AI identifies important topics and keywords

Spring Boot communicates with Python AI service using REST API calls.

9. Database Design

The database stores meeting information and AI output.

Main Tables

- User Table (id, name, email, password)
- Meeting Table (meeting_id, host_id, start_time, status)
- Participant Table (meeting_id, user_id)
- Transcript Table (meeting_id, transcript_text)
- Summary Table (meeting_id, summary_text)

This helps to maintain meeting history and store AI results.

10. Testing

Testing is performed using Postman and Swagger.

Tested Scenarios:

- User registration and login
- JWT token validation
- Meeting creation and joining
- Chat testing
- AI transcription output testing
- Summary generation testing

This ensures correct output and smooth working of the project.

11. Conclusion

This AI-powered video conferencing website is a modern communication platform built using Java Spring Boot for backend and Python AI services for transcription and summarization. It provides secure meeting creation, real-time video/audio calling, chat, and AI-based productivity features.

This project reduces manual effort by generating meeting transcripts and summaries automatically. It is scalable and can be enhanced further with cloud deployment and advanced AI features.

End-to-End Simple Flow:

User Login → JWT Token → Create Meeting → Meeting Code → Participants Join → WebRTC Call Start → Audio Stream → Python AI Transcription → Transcript → AI Summary → Meeting History Stored in Database